CTSYSINL · FINAL PROJECT

NEKAI

Technical Documentation

Containerized Infrastructure and CI/CD Automation using Docker and Jenkins

Course / Section	CTSYSINL – ITE305
Group name	Cacheless
Project	NEKAI, an anime watchlist with an AI companion (Neko)
Repository	https://github.com/jnncy-techcookie/nekai
Date	October 20, 2026

Group members and roles

Member	Role (guidelines, Section 3)
Gloria, Raiza Lhyn A.	Project Lead / Scrum Master
Alcoriza, Cheska Joyce S.	DevOps / CI-CD Engineer
David, Heidee S.	Infrastructure Engineer
Pugal, Jenna Cyrille G.	Backend and Database Engineer
Madronero, Kristine Mae M.	Frontend, QA and Documentation Lead

Contents

1. Project description and modules
2. Architecture
3. Technology stack
4. Dockerfiles
5. Docker Compose
6. Jenkins setup
7. Pipeline
8. Testing strategy
9. Rollback and data persistence

1. Project description and modules

NEKAI is an anime watchlist that works on both phones and desktops. After creating an account, you can build your own library, sorted into Watching, Plan to Watch, Completed and Dropped. You can log the episodes you've seen, rate titles, and earn XP, levels, streaks and achievement badges along the way. The app also comes with **Neko**, a chat companion powered by Google Gemini. Neko knows what's on your list, so you can ask it questions about your own watching. **Nekai's Picks** suggests new titles to try next. All the anime data (titles, posters, genres and episodes) comes from the Tenrai API, an unofficial provider of MyAnimeList data.

We split the application into three modules that we wrote ourselves, plus a reverse proxy and a database. Each one runs in its own container. Jenkins runs in a separate container from its own compose project, and it handles the CI/CD pipeline.

Container	What it does	Technology	Health endpoint
frontend	The part people see in the browser: six static pages (Home, Library, Discover, Profile, Settings and Sign in), the Neko chat panel and the anime detail panel. Every page also shows the deployed build number, read from /build.json, so we can always tell which version is live.	HTML, CSS and JavaScript, served by Nginx	GET /health
api	Handles accounts (sign up, sign in and password reset) and everything a user saves: their library, history, badges, settings, profile and Neko chats. It also stores profile pictures and fetches anime data from Tenrai through a single queue that is rate-limited and cached. It is the only module allowed to talk to the database.	Node.js 24, Express 5, pg	GET /api/health (returns 200 only when PostgreSQL responds)
ai-service	Runs Neko's chat and Nekai's Picks through Gemini. It applies a rate limit per user, switches to a fallback model when needed, and follows clear retry rules. It has no database of its own, so it asks the api to check the session cookie and to look up anime.	Node.js 24, Express 5	GET /health
proxy	The single way in from outside, and the only container with a published port. It sends / to the frontend, /api/neko and /api/recommend to the ai-service, and every other /api path plus /avatars to the api.	Nginx (unprivileged)	GET /health
db	A PostgreSQL database with 13 tables covering users, sessions, profiles, settings, the anime catalog, libraries, watch history, achievements, recommendations and Neko chats. The schema is created from db/init.sql the first time the database starts, and the data is kept in the pg-data named volume.	PostgreSQL 16	pg_isready

This setup covers what the project asks for. We have at least three of our own modules in separate containers (frontend, api and ai-service), the api reads from and writes to the database, every backend module has a health endpoint, and the pipeline runs automated tests for the api, the ai-service and the frontend.

2. Architecture

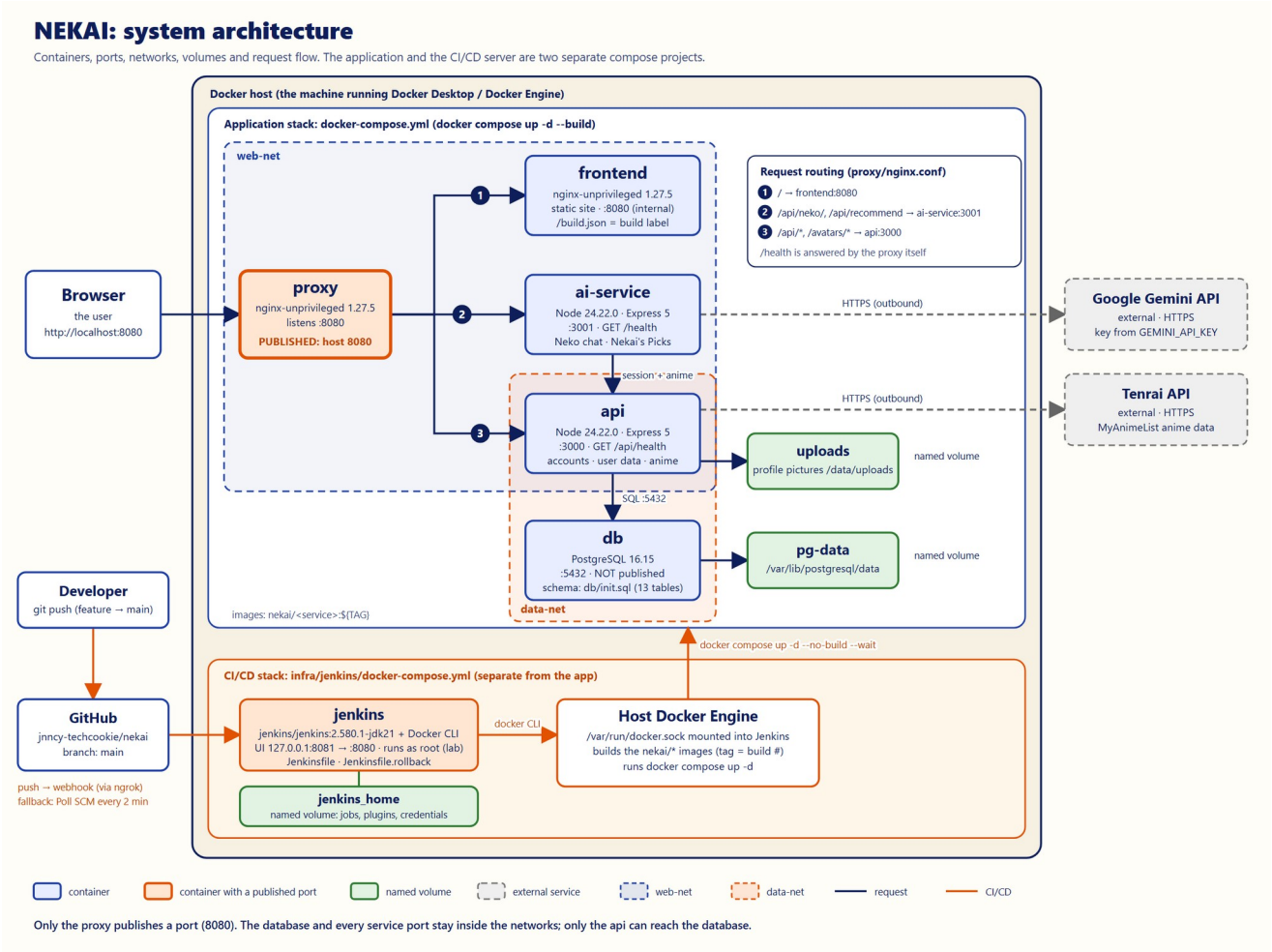


Figure 1. Containers, ports, networks, volumes and request flow (also saved as docs/architecture.png).

Containers, ports, networks and volumes

Container	Internal port	Published	Networks	Storage
proxy	8080	**host 8080** (HTTP_PORT)	web-net	none (the config is built into the image)
frontend	8080	no	web-net	none (the site is built into the image)
api	3000	no	web-net, data-net	uploads volume (profile pictures)
ai-service	3001	no	web-net	none (stateless)
db	5432	no	data-net	pg-data volume
jenkins (separate project)	8080	127.0.0.1:8081 only	the default network of its own project	jenkins_home volume, with the Docker socket mounted

Network separation. We use two networks. web-net connects the proxy, frontend, api and ai-service, while data-net connects only the api and the database. Because of this, the proxy, frontend and ai-service have no way to reach PostgreSQL, and the database port is never published to the host.

Request flows

1. **Opening a page.** The browser asks the proxy for /, and the proxy serves the static files from the frontend. Once the page loads, its scripts call /api/me.
2. **Loading and saving your data.** The browser calls /api/me through the proxy, which passes the request to the api, and the api reads from PostgreSQL. The api checks the session cookie first (we only store a SHA-256 hash of the token, never the token itself) and makes sure every query only touches the signed-in user's rows. When you save, all of your changes go in as one batch inside a single transaction.
3. **Asking Neko.** The browser sends the message to /api/neko/chat, and the proxy forwards it to the ai-service. The ai-service passes the cookie along to the api to find out who you are, then calls Gemini over HTTPS using the API key from its own environment. The key never reaches the browser.
4. **Searching for anime.** The browser calls /api/tenrai/..., and the proxy hands it to the api, which calls Tenrai. To stay within Tenrai's rate limit, the api queues and caches these requests for all users together.
5. **Deploying.** A git push reaches GitHub, which notifies Jenkins (through the ngrok webhook, or Jenkins finds the change itself with Poll SCM). Jenkins then talks to the host's Docker Engine through the mounted socket to build the images and update the running application (see Sections 6 and 7).

A few of these choices were made with security in mind. Only the proxy publishes a port, Jenkins' web interface is bound to 127.0.0.1, the services run as non-root users, and secrets are passed in when the containers start instead of being stored in the repository.

3. Technology stack

Component	Choice	Why we chose it
Containers	Docker, Docker Compose	The project requires it. Compose also lets us start the whole stack with one command and keep the dependencies, health checks, networks and volumes together in one file.
Reverse proxy and static server	Nginx 1.27.5 (nginx-unprivileged, Alpine)	It's small, fast and well documented, and the unprivileged image runs without root. By pointing Nginx at Docker's DNS resolver and using variables in proxy_pass, it looks up service names on every request. That means it finds a container again after a redeploy without the proxy needing a restart.
Backend language and framework	Node.js 24.22.0, Express 5	Using one language for both backend services and the frontend means everyone on the team can read all of the code. Node also has a built-in test runner (node --test), so our tests don't need any extra packages.
Database	PostgreSQL 16.15 (Alpine)	Our data is relational (users own libraries, histories, badges and chats), so foreign keys with on delete cascade, check constraints and transactions are a natural fit. We started out on a hosted database (Supabase) and later moved to a PostgreSQL container so that the data lives in a named volume we control.
Frontend	Plain HTML, CSS and JavaScript	There's no build step, so the image is just static files behind Nginx and the pipeline has nothing to compile.
AI	Google Gemini API	It has a free tier. We use it for Neko and Nekai's Picks, with a fallback model, and the app keeps working (with those features limited) when there's no key or the quota runs out.
Anime data	Tenrai API	It provides MyAnimeList data for free. Only the api calls it, through a shared queue and cache.

Component	Choice	Why we chose it
CI/CD server	Jenkins LTS 2.580.1 (JDK 21) in Docker	The project requires it. It gives us a declarative pipeline written as code, a Stage View that's easy to follow during the demo, and a safe place to store the production .env as a credential.
Git hosting and trigger	GitHub, with a webhook through ngrok and Poll SCM as a fallback	Jenkins runs on a laptop that has no public address. ngrok makes it reachable so webhooks arrive right away, and Poll SCM (every 2 minutes) keeps the pipeline automatic even if the tunnel or the network goes down.
Testing	node:test + node:assert	Both are built into Node. The tests run inside Docker as a test build stage, so the Jenkins host doesn't need Node installed at all.

4. Dockerfiles

Each module we wrote has its own Dockerfile and its own .dockerignore. For the database, we start from the official PostgreSQL image and add our schema on top of it.

Image	Stages and base images	Runs as	Notes
api	base → test → production node:24.22.0-alpine	node	Production installs packages with npm ci --omit=dev. The upload directory is created and owned by node so the mounted volume can be written to. HEALTHCHECK on /api/health.
ai-service	base → test → production node:24.22.0-alpine	node	Follows the same pattern as the api, with a HEALTHCHECK on /health.
frontend	test (node:24.22.0-alpine) → production (nginx-unprivileged:1.27.5-alpine)	nginx	The BUILD_NUMBER build argument is written into /build.json. The config is copied in, and the test files are left out of the final image.
proxy	single stage nginx-unprivileged:1.27.5-alpine	nginx	The routing config is built into the image, so no bind mount is needed.
db	postgres:16.15-alpine	the official entrypoint switches to postgres	init.sql is copied into /docker-entrypoint-initdb.d. It only runs when the data volume is empty, so existing data is never touched.

Practices we followed

- **Pinned, slim base images.** We use Alpine variants with exact version tags (for example node:24.22.0-alpine), so every build starts from the same base. To update one, we change the tag and let the pipeline's tests confirm nothing broke.
- **Layer caching.** package*.json is copied and installed before the rest of the source, so dependencies are only reinstalled when they actually change.
- **Multi-stage builds with a test stage.** Running docker build --target test ./api runs the tests inside Docker. If a test fails, the build fails, and that's exactly what Jenkins relies on as its quality gate. The production stage doesn't include dev dependencies or test files.
- **No root at runtime.** The Node services run as the node user, and both Nginx images are the unprivileged variants (port 8080, no root).

- **.dockerignore** in every module keeps out node_modules, .env files, uploaded data and the Dockerfile itself.
- **HEALTHCHECK** in every service image. Compose uses these for its depends_on conditions and for docker compose up --wait.
- **Config built into the images** instead of bind mounts. This avoids problems with host paths when Jenkins, which itself runs in a container, starts Compose.

```
# api/Dockerfile (excerpt): the test stage is the pipeline's quality gate
FROM node:24.22.0-alpine AS base
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .

FROM base AS test
RUN npm test          # a failing test fails "docker build --target test"

FROM node:24.22.0-alpine AS production
...
USER node
HEALTHCHECK --interval=15s --timeout=3s --start-period=10s --retries=3 \
  CMD wget -qO- http://127.0.0.1:3000/api/health || exit 1
```

5. Docker Compose

docker-compose.yml (project name nekai) starts the whole application with a single docker compose up -d --build. We kept Jenkins out of this file on purpose, as explained in Section 6.

Services, dependencies and health checks

Service	Waits for (service_healthy)	Health check
db	nothing	pg_isready -U $POSTGRES_USER -d $POSTGRES_DB every 10 s, with 10 retries and a 10 s start period
api	db	wget /api/health every 15 s (returns 200 only when PostgreSQL can be reached)
ai-service	api	wget /health every 15 s
frontend	nothing	wget /health every 15 s
proxy	frontend, api, ai-service	wget /health every 15 s

So the services start in the order **db** → **api** → **ai-service** → **proxy**, while the frontend starts alongside them. Every service is set to restart: unless-stopped.

Networks and volumes

- **Networks:** two user-defined networks, web-net (proxy, frontend, api and ai-service) and data-net (api and db).
- **Volumes:** pg-data holds every account, list, history entry and chat, and uploads holds the profile pictures. Running docker compose down and then up keeps both. Only down -v deletes them, and the pipeline never uses -v.
- **Published ports:** only the proxy (${HTTP_PORT:-8080}:8080).

Environment variables and secrets

All of the configuration comes from environment variables. The real .env is never committed because it's listed in .gitignore, and .env.example holds placeholder values instead. In Jenkins, the production

.env is stored as a **Secret file** credential (nekai-env) and handed to Compose with --env-file, so it never gets copied into the workspace.

Variable	Default / rule	Used by
DB_USER, DB_PASSWORD	**required** (${VAR:?})	db (creates the user) and api (PGUSER, PGPASSWORD)
DB_NAME	nekai	db, api
HTTP_PORT	8080	the proxy's published port
TAG	latest, or the build number when Jenkins sets it	image tags nekai/<service>:${TAG} and the build label on the page
GEMINI_API_KEY, GEMINI_MODEL, GEMINI_FALLBACK_MODEL	optional; the app still works without a key	ai-service
APP_URL, SMTP_*, MAIL_FROM	optional; without SMTP, the reset email is printed to the api log instead	api (password reset links)

Images are named nekai/<service>:${TAG}. Since every deploy keeps its own build-numbered images, we can start an earlier version again without rebuilding anything (see Section 9).

6. Jenkins setup

Installation

Jenkins runs in Docker from its own compose project in infra/jenkins/. Keeping it separate means the pipeline can never accidentally redeploy Jenkins itself.

- **Image** (infra/jenkins/Dockerfile): we start from jenkins/jenkins:2.580.1-jdk21 and add the Docker CLI and the Compose plugin from Docker's apt repository. The plugins are installed into the image with jenkins-plugin-cli, using pinned versions from plugins.txt, so the setup wizard has nothing left to download. Build arguments let us switch to a mirror when jenkins.io can't be reached.
- **Compose** (infra/jenkins/docker-compose.yml): the jenkins_home named volume keeps the jobs, plugins and credentials. The host's /var/run/docker.sock is mounted so the pipeline can build and deploy. The UI is published on 127.0.0.1:8081, which means it can only be opened from the same computer (and through ngrok for the webhook).
- **Starting it:** run cd infra/jenkins && docker compose up -d --build, then docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword to get the unlock password. To confirm Jenkins can reach Docker, run docker exec jenkins docker ps.

Plugins (pinned in plugins.txt): Pipeline (workflow-aggregator), Git, GitHub, GitHub Branch Source, Credentials Binding, Docker Pipeline, Pipeline Stage View, Timestamper, Workspace Cleanup, Build Timeout, Folders, OWASP Markup Formatter and Matrix Authorization.

Credentials

Only one credential is needed: a **Secret file** with the ID nekai-env that contains the production .env (in the same format as .env.example). The Jenkinsfile refers to it by ID through withCredentials, so no secret ever shows up in the repository. We would only need a GitHub access token credential if the repository were private.

Jobs

Job	Definition	Purpose
nekai	Pipeline script from SCM, GitHub repository, branch */main, Script Path Jenkinsfile	Tests, builds and deploys on every push
nekai-rollback	Pipeline script from SCM, same repository, Script Path Jenkinsfile.rollback	A manual rollback that takes a build number as a parameter (Section 9)

Trigger configuration

The Jenkinsfile declares **two** triggers, so the pipeline starts on its own and nobody has to press Build Now:

- **Webhook (instant):** githubPush(). We expose Jenkins with ngrok http 8081, and the GitHub repository has a webhook (Settings → Webhooks) with the Payload URL https://<ngrok-url>/github-webhook/, content type application/json, sending push events.
- **Poll SCM (fallback):** pollSCM('H/2 * * * *') checks GitHub every two minutes. It doesn't need a public URL, so the pipeline stays automatic even when the tunnel or the network is down.

Jenkins only registers the triggers in a Jenkinsfile after the job has run once, so we run the job once by hand right after creating it.

Security note. Mounting /var/run/docker.sock and running Jenkins as root gives Jenkins full control over the host's Docker. That's acceptable on a lab laptop, but it wouldn't be in production. Safer options would be dedicated build agents, rootless Docker, Docker-in-Docker over TLS, or daemonless builders such as Kaniko or BuildKit. To reduce the risk in our setup, the Jenkins port is bound to 127.0.0.1 and no secrets are stored in the repository.

7. Pipeline

The declarative Jenkinsfile in the root of the repository defines the pipeline as code. It sets COMPOSE_PROJECT_NAME=nekai and TAG=${BUILD_NUMBER}, allows only one build at a time (disableConcurrentBuilds), keeps the last 15 builds, and stops any build that runs longer than 20 minutes.

Stage	What it does	If it fails
Checkout	checkout scm pulls the commit that triggered the build.	The pipeline stops and nothing changes.
Test	Runs docker build --no-cache --target test for the api, the ai-service and the frontend, so the tests run as part of the build.	**Quality gate.** The pipeline stops here. No production image gets built and nothing is deployed, so the running app stays on its previous version.
Build Images	Runs docker compose build with the nekai-env secret file and tags the images with the build number.	The pipeline stops, and the running containers aren't touched.
Deploy	Runs docker compose up -d --no-build --wait --wait-timeout 120. Compose only recreates the services that changed and waits until every health check passes. The volumes are kept.	The build fails. The new containers may be unhealthy, so the fix is to run the rollback job.

Stage	What it does	If it fails
Smoke Test / Verify	Runs scripts/smoke-test.js inside the api container. It checks the proxy's /health, the frontend page, /api/health (which proves the database connection works), the ai-service's /health, AI routing through the proxy, and that /build.json reports *this* build's number. It keeps retrying for up to a minute while the services settle.	The build fails, and the fix is to run the rollback job.

After the run. When a build succeeds, Jenkins reports that build N is deployed and verified. When it fails, it tells you which stage to look at and points you to the nekai-rollback job. The rollback pipeline uses the same smoke-test script.

8. Testing strategy

Suite	Tests	What it checks
api	43	**Unit:** the rate limiter, scrypt password hashing (salted, and the plain password is never stored), session tokens (random, and only a SHA-256 hash is stored), and the rules for what a save may contain. **Routes:** the health check (503 when the database is down), session handling, input checks on sign-up and password reset, bad or oversized JSON, and the Tenrai routes (invalid filters are refused, only safe-for-work results are returned, and upstream errors are mapped). **Sessions:** sign-up and sign-in, a 409 for an email that's already registered, saves that run in one transaction and only touch the user's own rows, and profile pictures that can only point to the user's own file. **Server:** JSON 404 responses, malformed JSON, and no X-Powered-By header.
ai-service	22	The session check through the api (401 without a cookie), validation of Neko requests, how Gemini is called (the key goes in a header, 15 messages a minute per user, falling back to the second model when the quota runs out, retrying on 503 but not on 400, and handling empty answers), Nekai's Picks (watch-history validation, dropping titles already on the list, and 4 refreshes a minute), plus health and 404.
frontend	6	Every script parses, every local src/href, data-app script, CSS @import and url() points to a file that exists, and nginx.conf serves /health and /build.json.

How the tests run and how a failure blocks deployment

- The tests live in each service's Dockerfile as a test stage. The Jenkins **Test** stage builds those stages with --no-cache. If even one test fails, docker build exits with an error and the stage fails. None of the later stages run, so broken code never gets built for deployment or deployed.
- The tests need **no database and no internet connection**. We replace the database layer and outgoing fetch calls (to Tenrai and Gemini) with test doubles, so the tests are fast and give the same result every time. Anyone can also run them on their own laptop with npm test inside the service folder.
- **Smoke test versus unit tests.** The unit tests check the code before it ships. The smoke test checks the real, running system after the deploy: routing through the proxy, the live database connection, the AI route and the correct build label.

Quality gate demonstration

We added a test to the api specifically to demonstrate the quality gate (api/tests/server.test.js, which checks that unknown API routes return a JSON 404). If we change the status it expects and push, the

build fails at Test, and the site keeps serving the previous build. Reverting the change makes the next build pass again. You can see this demonstration in the repository history in the commits "Demonstrate failing test blocks deployment" and "Restore correct API test expectation".

9. Rollback and data persistence

Rollback procedure

The images from every build stay on the host, tagged with the build number (nekai/api:14, nekai/db:14, and so on). That means a rollback doesn't rebuild anything. It simply switches the running stack back to an earlier build's images. We do this with the Jenkins job nekai-rollback (Jenkinsfile.rollback), which you start with **Build with Parameters** and a ROLLBACK_TAG parameter (a build number such as 14).

1. **Validate.** The tag has to contain only digits, and all five images (proxy, frontend, api, ai-service and db) have to exist for that build. If they don't, the log lists the builds that are available and nothing is changed.
2. **Rollback.** Runs docker compose up -d --no-build --wait with TAG set to the chosen build, using the same nekai-env credential.
3. **Smoke Test / Verify.** Runs the same checks as a normal deploy, and also checks that the page reports the build number we rolled back to.

You can get the same result without Jenkins by running one command on the host: TAG=14 docker compose --env-file <.env> up -d --no-build. Either way, the build label in the corner of every page shows which build is live.

Limits. A rollback doesn't touch the volumes, so it changes the code but not the data. The job uses the current docker-compose.yml. If an image has been removed with docker image prune, we can't roll back to it anymore. The next push simply deploys a new build on top of the rolled-back one.

Data persistence

Every account, list, history entry, badge and chat is stored in PostgreSQL, inside the pg-data named volume, and profile pictures are stored in the uploads volume. Both survive a redeploy through the pipeline and a docker compose down followed by up -d, because neither of those removes volumes. The schema script only runs when the volume is empty, so an existing database is never changed.

How we prove it:

1. Sign in and add some titles to the library.
2. Run docker compose down (without -v) and then docker compose up -d, or push a change and let the pipeline redeploy.
3. Sign in again and check that the library is the same. Throughout all of this, docker volume ls keeps listing nekai_pg-data and nekai_uploads.